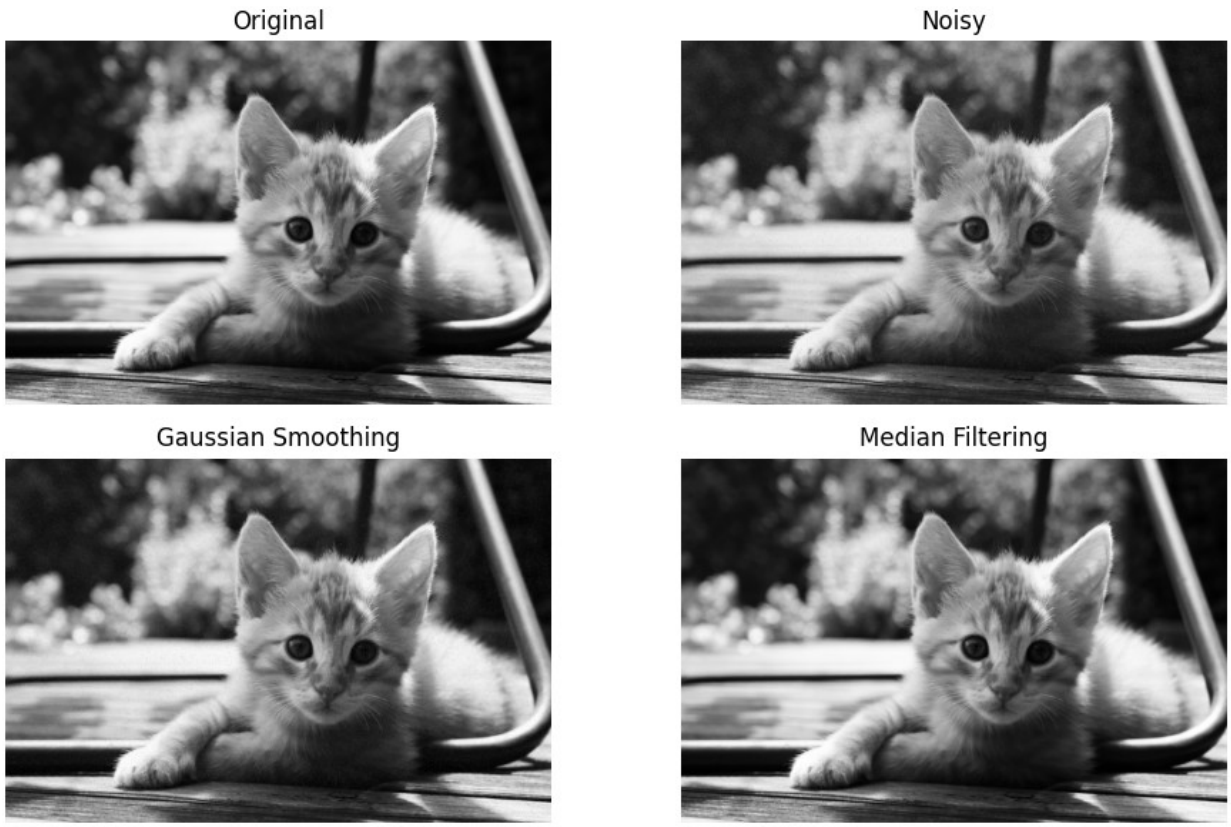
```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

# ----- Add artificial salt-and-pepper noise for demo -----
noisy = img.copy()
p = 0.02  # noise probability
rnd = np.random.rand(*img.shape)
noisy[rnd < p/2] = 0       # pepper
noisy[rnd > 1 - p/2] = 255 # salt

# 1) Smoothing with Gaussian blur
gaussian = cv2.GaussianBlur(noisy, (5, 5), 1.0)

# 2) Median filtering
median = cv2.medianBlur(noisy, 5)

# Show results
titles = ["Original", "Noisy", "Gaussian Smoothing", "Median
Filtering"]
images = [img, noisy, gaussian, median]

plt.figure(figsize=(10, 6))

for i in range(4):
    plt.subplot(2, 2, i+1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()
```




```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving input.jpg.jpg to input.jpg (1).jpg
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

# Check if image loaded properly
if img is None:
    print("Error: Image not loaded. Check the file name.")
else:
    # 1) Histogram Equalization
    hist_eq = cv2.equalizeHist(img)

    # 2) Contrast Stretching (Min-Max Normalization)
    min_val = np.min(img)
    max_val = np.max(img)
```

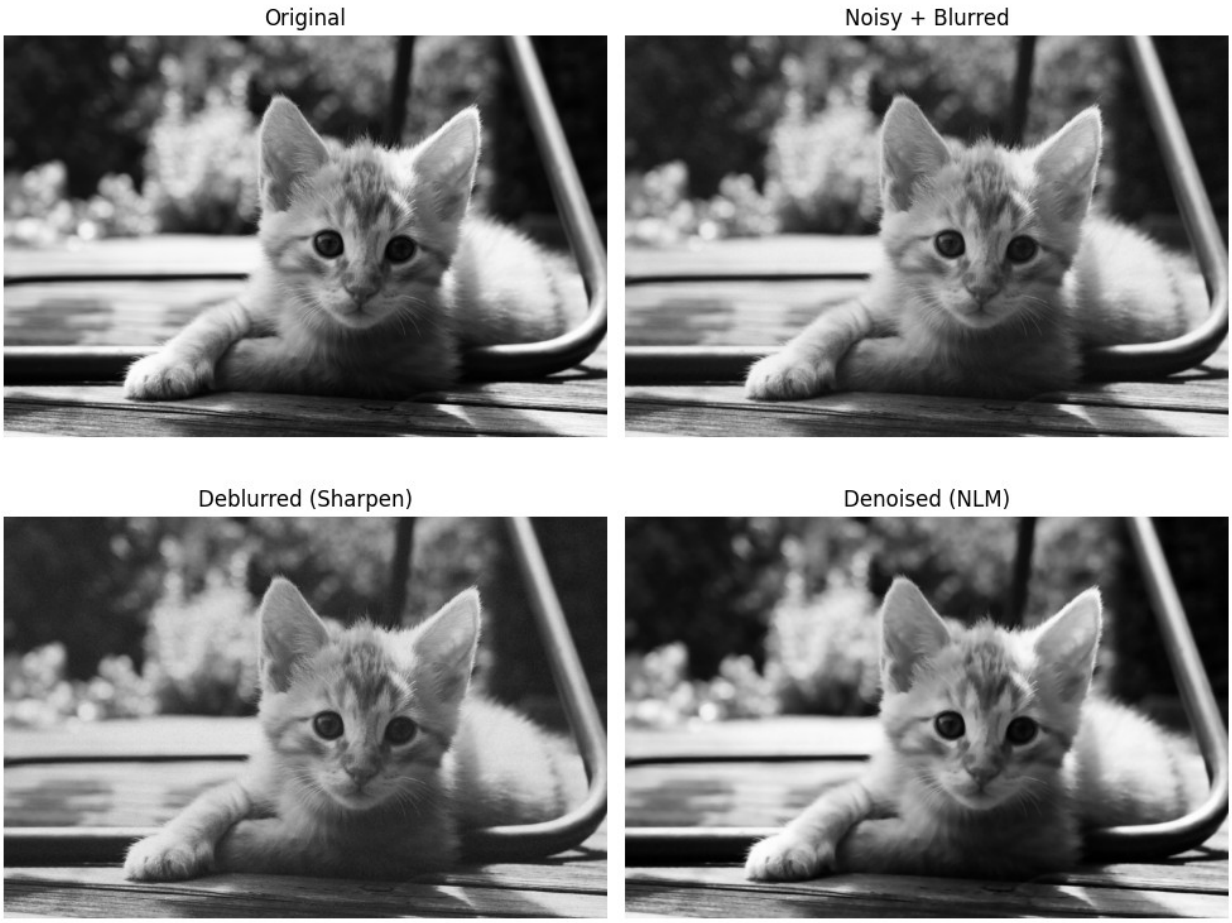
```
    # Avoid division by zero
    if max_val - min_val == 0:
        contrast_stretched = img.copy()
    else:
        contrast_stretched = ((img - min_val) *
                              (255.0 / (max_val -
min_val))).astype(np.uint8)

    # Show results
    titles = ["Original", "Histogram Equalization", "Contrast
Stretching"]
    images = [img, hist_eq, contrast_stretched]

    plt.figure(figsize=(12, 4))

    for i in range(3):
        plt.subplot(1, 3, i+1)
        plt.imshow(images[i], cmap="gray")
        plt.title(titles[i])
        plt.axis("off")

    plt.tight_layout()
    plt.show()
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

# Check if image loaded
if img is None:
    print("Error: Image not loaded. Check file name.")
else:
    # Slight blur to reduce noise before edge detection
    blur = cv2.GaussianBlur(img, (3, 3), 0)

    # 1) Sobel edges
    sobelx = cv2.Sobel(blur, cv2.CV_64F, 1, 0, ksize=3)
```

```
    sobely = cv2.Sobel(blur, cv2.CV_64F, 0, 1, ksize=3)

    sobel_mag = cv2.magnitude(sobelx, sobely)
    sobel_mag = cv2.convertScaleAbs(sobel_mag)

    # 2) Canny edges
    canny_edges = cv2.Canny(blur, threshold1=100, threshold2=200)

    # Show results
    titles = ["Original", "Sobel Magnitude", "Canny Edges"]
    images = [img, sobel_mag, canny_edges]

    plt.figure(figsize=(12, 4))

    for i in range(3):
        plt.subplot(1, 3, i+1)
        plt.imshow(images[i], cmap="gray")
        plt.title(titles[i])
        plt.axis("off")

    plt.tight_layout()
    plt.show()
```

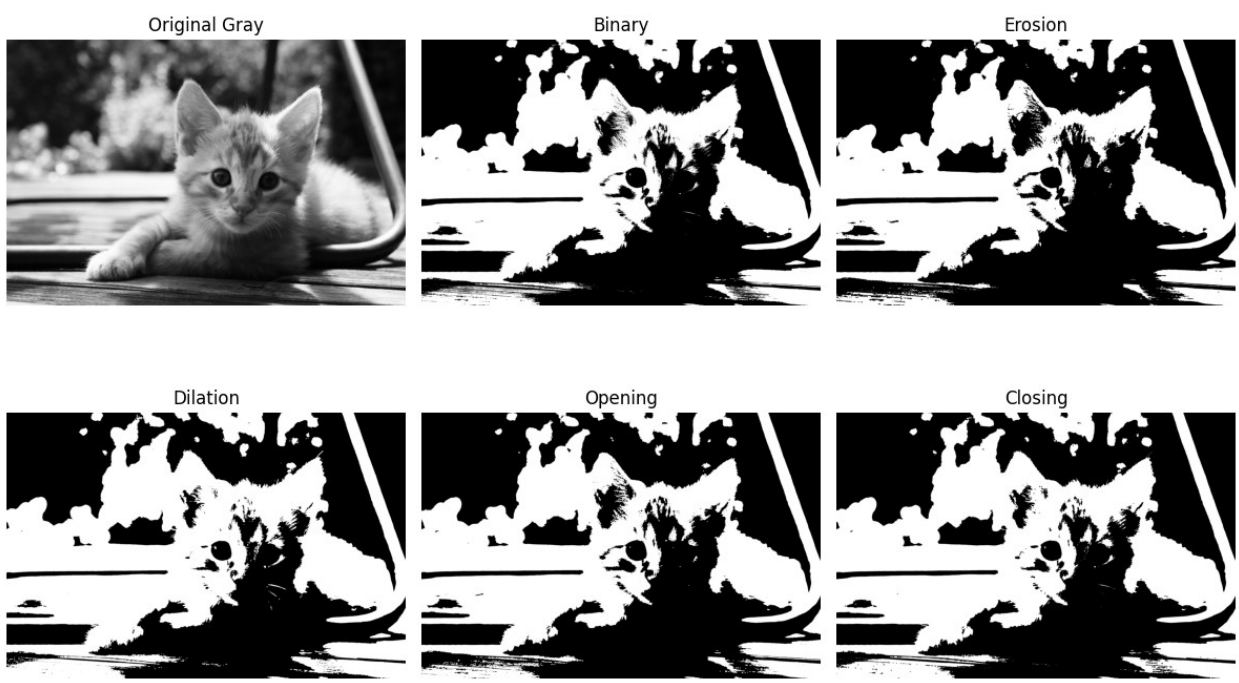
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read grayscale image
img = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

# Check if image loaded
if img is None:
    print("Error: Image not loaded. Check file name.")
else:
    # ----- Create blurred + noisy version -----
    blurred = cv2.GaussianBlur(img, (7, 7), 1.5)

    noise = np.random.normal(0, 10, img.shape).astype(np.float32)
    noisy_blurred = blurred.astype(np.float32) + noise
```

```
    noisy_blurred = np.clip(noisy_blurred, 0, 255).astype(np.uint8)

    # 1) Deblurring using sharpening kernel
    sharpen_kernel = np.array([[0, -1, 0],
                               [-1, 5, -1],
                               [0, -1, 0]], dtype=np.float32)

    deblurred = cv2.filter2D(noisy_blurred, -1, sharpen_kernel)

    # 2) Denoising using Non-Local Means
    denoised = cv2.fastNlMeansDenoising(noisy_blurred,
                                        None,
                                        h=15,
                                        templateWindowSize=7,
                                        searchWindowSize=21)

    # Show results
    titles = ["Original", "Noisy + Blurred",
              "Deblurred (Sharpen)", "Denoised (NLM)"]
    images = [img, noisy_blurred, deblurred, denoised]

    plt.figure(figsize=(10, 8))

    for i in range(4):
        plt.subplot(2, 2, i+1)
        plt.imshow(images[i], cmap="gray")
        plt.title(titles[i])
        plt.axis("off")

    plt.tight_layout()
    plt.show()
```




```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
img = cv2.imread("input.jpg", cv2.IMREAD_GRAYSCALE)

# Check if image loaded
if img is None:
    print("Error: Image not loaded. Check file name.")
else:
    # Threshold to get binary image (Otsu's method)
    _, binary = cv2.threshold(img, 0, 255,
                              cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # Structuring element (kernel)
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))

    # 1) Erosion
    eroded = cv2.erode(binary, kernel, iterations=1)

    # 2) Dilation
```

```
    dilated = cv2.dilate(binary, kernel, iterations=1)

    # 3) Opening (Erosion → Dilation)
    opened = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)

    # 4) Closing (Dilation → Erosion)
    closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

    # Display results
    titles = ["Original Gray", "Binary",
              "Erosion", "Dilation",
              "Opening", "Closing"]

    images = [img, binary, eroded,
              dilated, opened, closed]

    plt.figure(figsize=(12, 8))

    for i in range(6):
        plt.subplot(2, 3, i+1)
        plt.imshow(images[i], cmap="gray")
        plt.title(titles[i])
        plt.axis("off")

    plt.tight_layout()
    plt.show()
```




```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving test.jpg.png to test.jpg.png
```

```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving test.jpg.png to test.jpg (1).png
```

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

big = cv2.resize(img,None,fx=2,fy=2)
small = cv2.resize(img,None,fx=0.5,fy=0.5)

cv2_imshow(img)
cv2_imshow(big)
cv2_imshow(small)
```

```
Output hidden; open in https://colab.research.google.com to view.
```

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

h,w = img.shape[:2]

M1 = cv2.getRotationMatrix2D((w/2,h/2),45,1)
M2 = cv2.getRotationMatrix2D((w/2,h/2),-45,1)

r1 = cv2.warpAffine(img,M1,(w,h))
r2 = cv2.warpAffine(img,M2,(w,h))

cv2_imshow(r1)
cv2_imshow(r2)
```

```
Output hidden; open in https://colab.research.google.com to view.
```

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

M = np.float32([[1,0,100],[0,1,50]])

move = cv2.warpAffine(img,M,(img.shape[1],img.shape[0]))
```

```
cv2_imshow(img)
cv2_imshow(move)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

p1 = np.float32([[50,50],[200,50],[50,200]])
p2 = np.float32([[10,100],[200,50],[100,250]])

M = cv2.getAffineTransform(p1,p2)

out = cv2.warpAffine(img,M,(300,300))

cv2_imshow(img)
cv2_imshow(out)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

p1 = np.float32([[0,0],[300,0],[0,300],[300,300]])
p2 = np.float32([[50,50],[250,0],[0,250],[300,300]])

M = cv2.getPerspectiveTransform(p1,p2)

out = cv2.warpPerspective(img,M,(300,300))

cv2_imshow(img)
cv2_imshow(out)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

cap = cv2.VideoCapture(list(uploaded.keys())[0])

p1 = np.float32([[0,0],[300,0],[0,300],[300,300]])
p2 = np.float32([[50,50],[250,0],[0,250],[300,300]])

M = cv2.getPerspectiveTransform(p1,p2)
```

```
ret,frame = cap.read()

out = cv2.warpPerspective(frame,M,(300,300))

cv2_imshow(out)
```

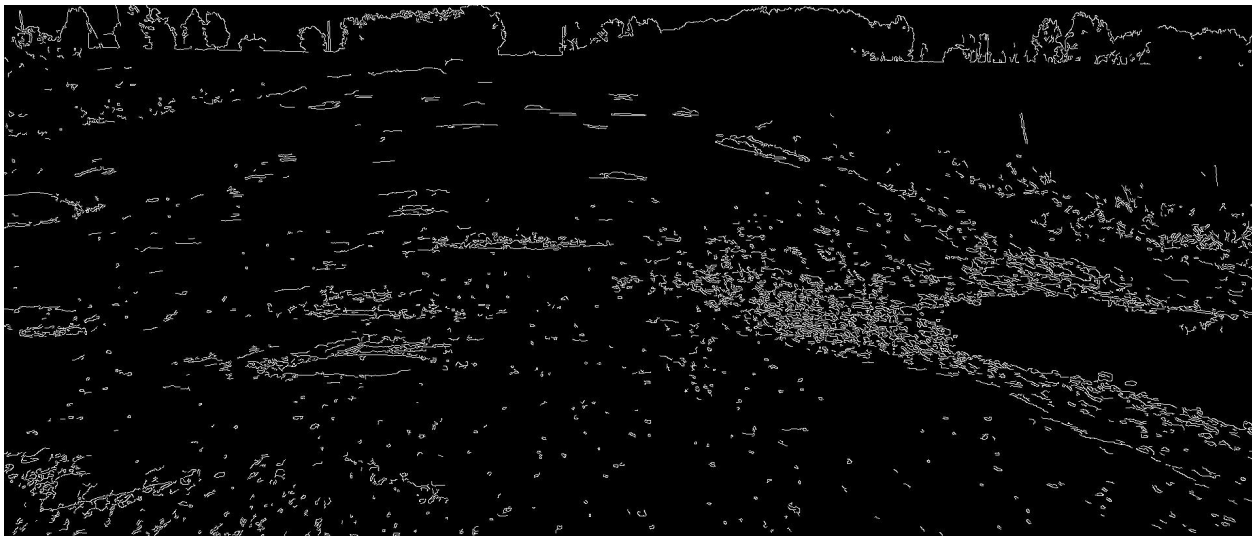
```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

p1 = np.float32([[10,10],[200,20],[20,200],[220,220]])
p2 = np.float32([[0,0],[300,0],[0,300],[300,300]])

H,_ = cv2.findHomography(p1,p2)

out = cv2.warpPerspective(img,H,(300,300))

cv2_imshow(img)
cv2_imshow(out)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

p1 = np.float32([[10,10],[200,20],[20,200],[220,220]])
p2 = np.float32([[0,0],[300,0],[0,300],[300,300]])

H = cv2.getPerspectiveTransform(p1,p2)

out = cv2.warpPerspective(img,H,(300,300))

cv2_imshow(img)
cv2_imshow(out)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

edge = cv2.Canny(img,100,200)

cv2_imshow(img)
cv2_imshow(edge)
```



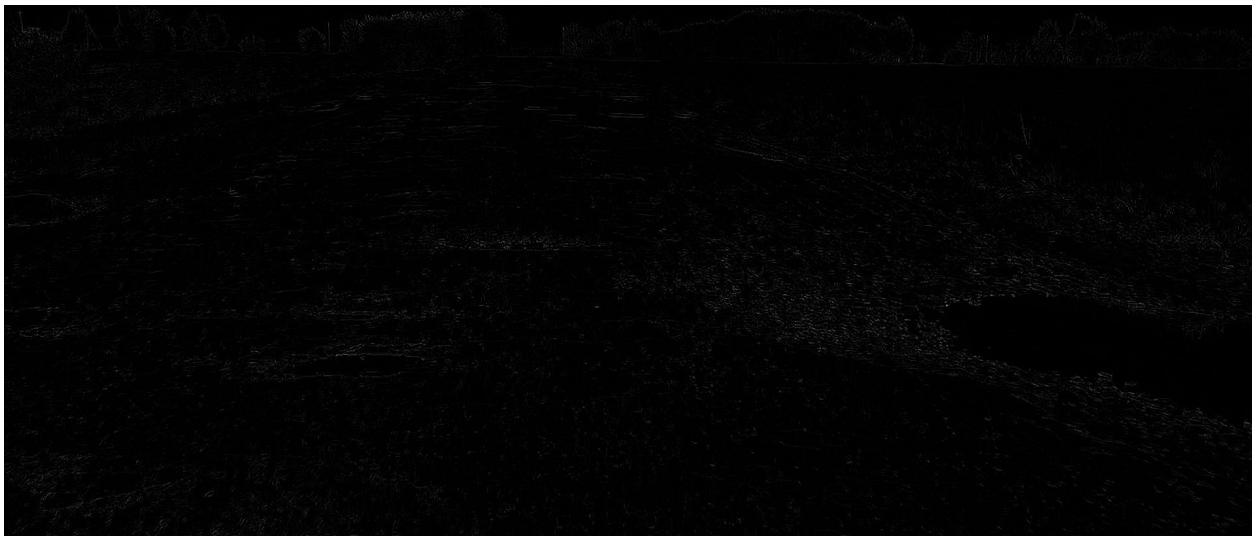
```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

sx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3)

cv2_imshow(img)
cv2_imshow(sx)
```

Output hidden; open in https://colab.research.google.com to view.

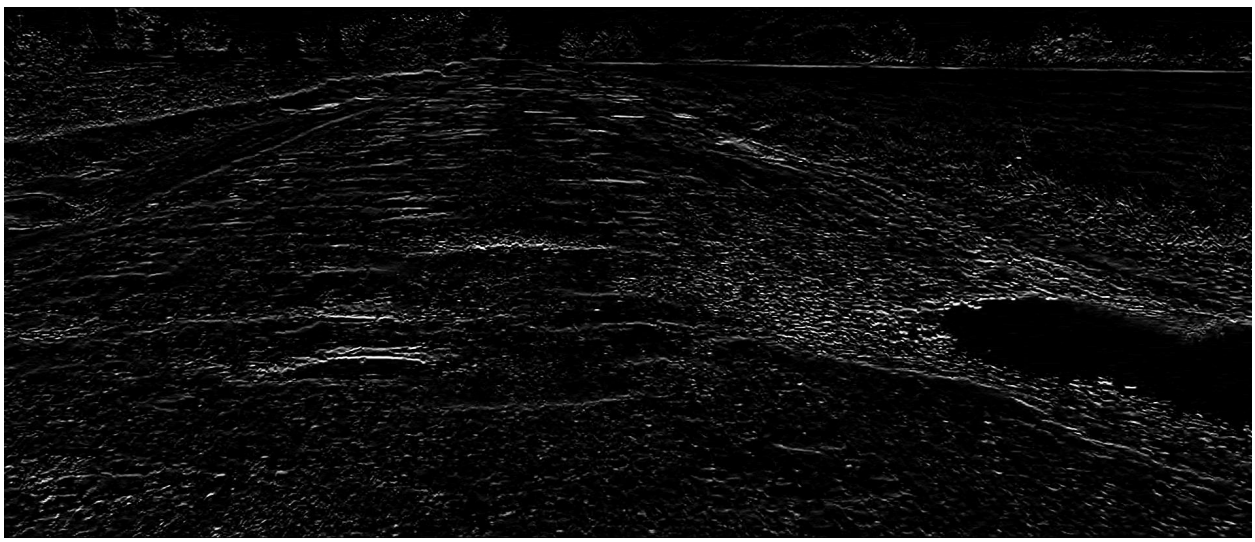
```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

sxy = cv2.Sobel(img,cv2.CV_64F,1,1,ksize=3)

cv2_imshow(img)
cv2_imshow(sxy)
```

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

sxy = cv2.Sobel(img,cv2.CV_64F,1,1,ksize=3)

cv2_imshow(img)
cv2_imshow(sxy)
```

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.array([[0,1,0],
                   [1,-4,1],
                   [0,1,0]])

sharp = cv2.filter2D(img,-1,kernel)

cv2_imshow(img)
cv2_imshow(sharp)
```



```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

sy = cv2.Sobel(img,cv2.CV_64F,0,1,ksize=3)

cv2_imshow(img)
cv2_imshow(sy)
```



```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.array([[1,1,1],
                   [1,-8,1],
                   [1,1,1]])

sharp = cv2.filter2D(img,-1,kernel)

cv2_imshow(img)
cv2_imshow(sharp)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.array([[0,-1,0],
                   [-1,5,-1],
                   [0,-1,0]])

sharp = cv2.filter2D(img,-1,kernel)

cv2_imshow(img)
cv2_imshow(sharp)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

blur = cv2.GaussianBlur(img,(5,5),0)

sharp = cv2.subtract(img,blur)

cv2_imshow(img)
cv2_imshow(sharp)
```

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

blur = cv2.GaussianBlur(img,(5,5),0)

A = 2

mask = cv2.subtract(img,blur)

highboost = cv2.add(img,A*mask)

cv2_imshow(img)
cv2_imshow(highboost)
```

```
Output hidden; open in https://colab.research.google.com to view.
```

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

gx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=3)
gy = cv2.Sobel(img,cv2.CV_64F,0,1,ksize=3)

grad = cv2.add(gx,gy)

cv2_imshow(img)
cv2_imshow(grad)
```

```
Output hidden; open in https://colab.research.google.com to view.
```

```
import cv2
from google.colab.patches import cv2_imshow
```

```
img = cv2.imread(list(uploaded.keys())[0])

watermark = img.copy()

cv2.putText(watermark,"WATERMARK",(50,50),cv2.FONT_HERSHEY_SIMPLEX,1,
(0,0,255),2)

cv2_imshow(img)
cv2_imshow(watermark)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

crop = img[50:200,50:200]

img[250:400,250:400] = crop

cv2_imshow(img)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.array([[-1,-1,-1],
                   [-1,8,-1],
                   [-1,-1,-1]])

boundary = cv2.filter2D(img,-1,kernel)

cv2_imshow(img)
cv2_imshow(boundary)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.ones((5,5),np.uint8)

erosion = cv2.erode(img,kernel)
```

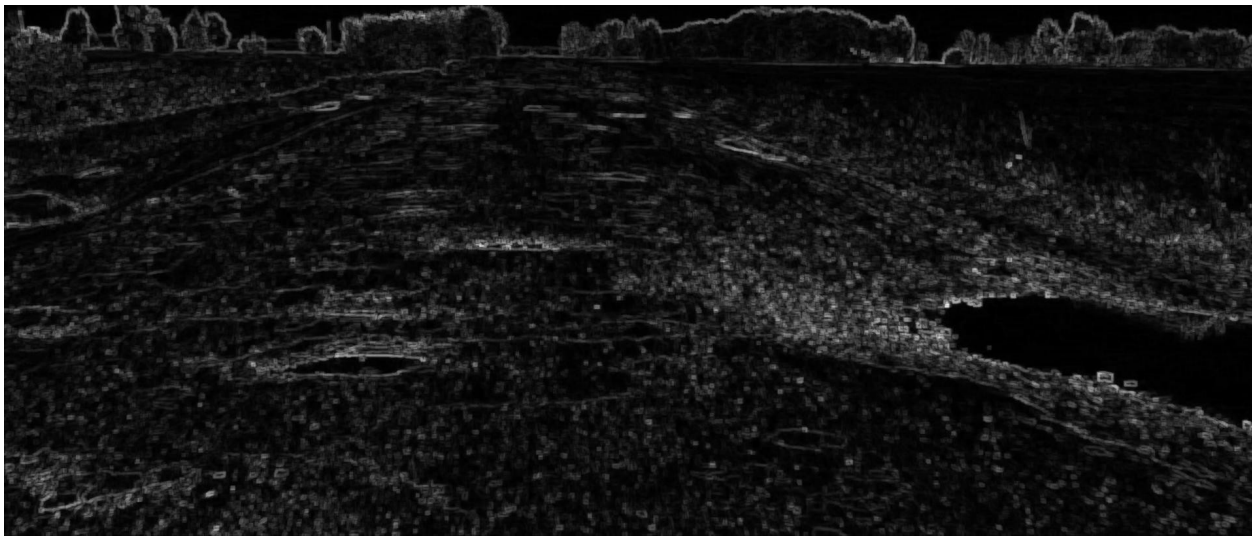
```
cv2_imshow(img)
cv2_imshow(erosion)
```

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.ones((5,5),np.uint8)

dilation = cv2.dilate(img,kernel)

cv2_imshow(img)
cv2_imshow(dilation)
```

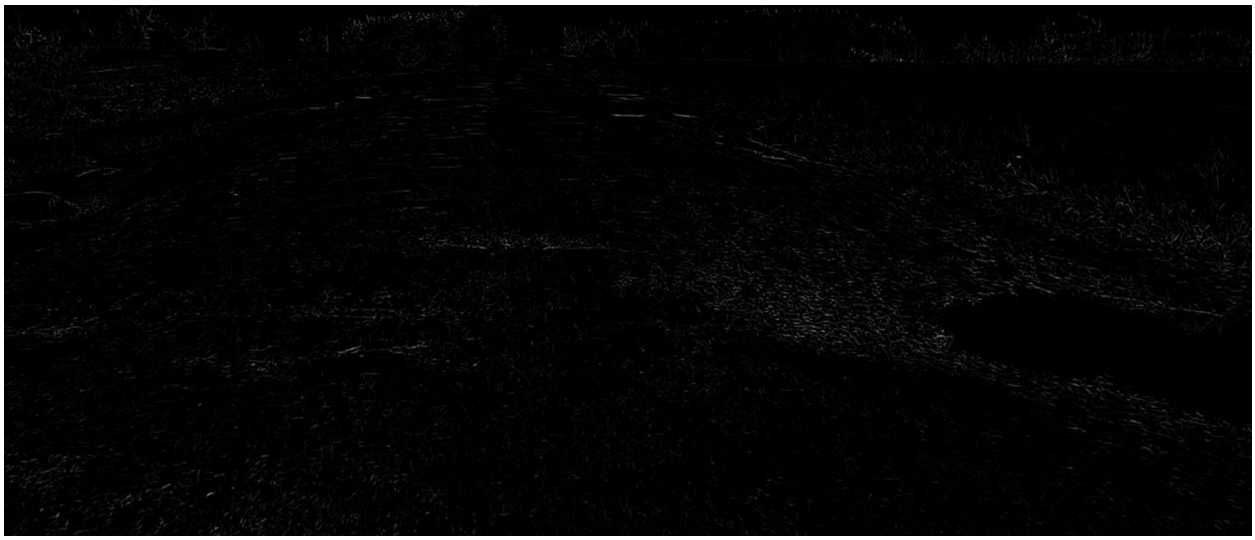
```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.ones((5,5),np.uint8)

opening = cv2.morphologyEx(img,cv2.MORPH_OPEN,kernel)

cv2_imshow(img)
cv2_imshow(opening)
```

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.ones((5,5),np.uint8)

closing = cv2.morphologyEx(img,cv2.MORPH_CLOSE,kernel)

cv2_imshow(img)
cv2_imshow(closing)
```

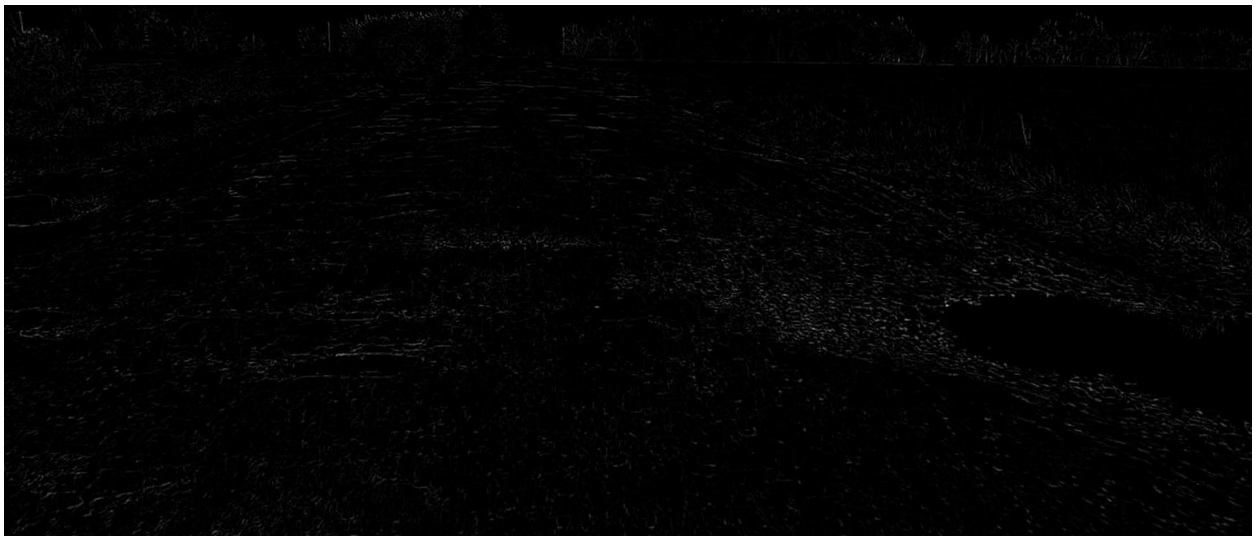
```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.ones((5,5),np.uint8)

gradient = cv2.morphologyEx(img,cv2.MORPH_GRADIENT,kernel)

cv2_imshow(img)
cv2_imshow(gradient)
```



```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.ones((5,5),np.uint8)

tophat = cv2.morphologyEx(img,cv2.MORPH_TOPHAT,kernel)

cv2_imshow(img)
cv2_imshow(tophat)
```



```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0],0)

kernel = np.ones((5,5),np.uint8)

blackhat = cv2.morphologyEx(img,cv2.MORPH_BLACKHAT,kernel)

cv2_imshow(img)
cv2_imshow(blackhat)
```



```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving Screenshot 2026-03-11 125515.png to Screenshot 2026-03-11
125515.png
```

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

circles = cv2.HoughCircles(gray,cv2.HOUGH_GRADIENT,1,100,

param1=50,param2=30,minRadius=50,maxRadius=200)
```

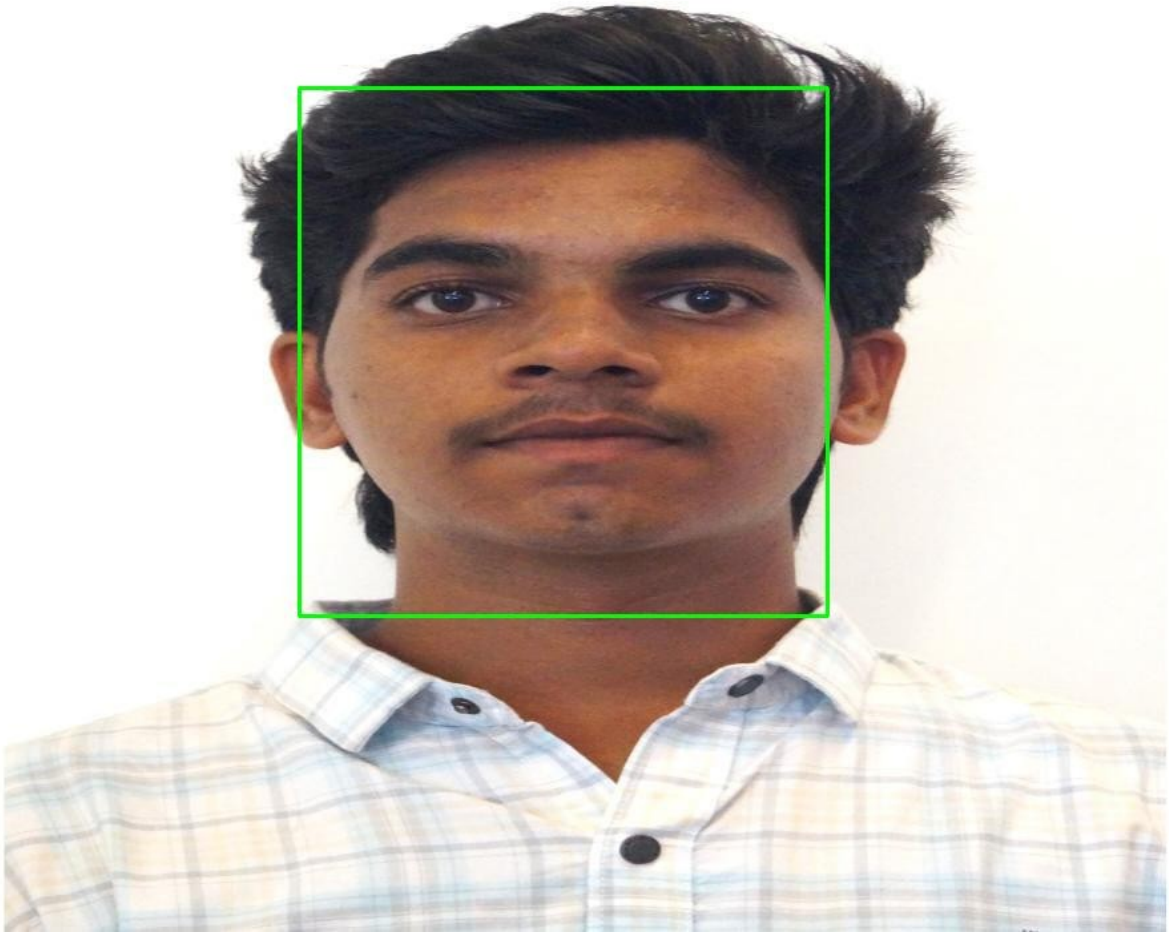
```
if circles is not None:
    circles = circles[0].astype(int)
    for x,y,r in circles:
        cv2.circle(img,(x,y),r,(0,255,0),3)

cv2_imshow(img)
```

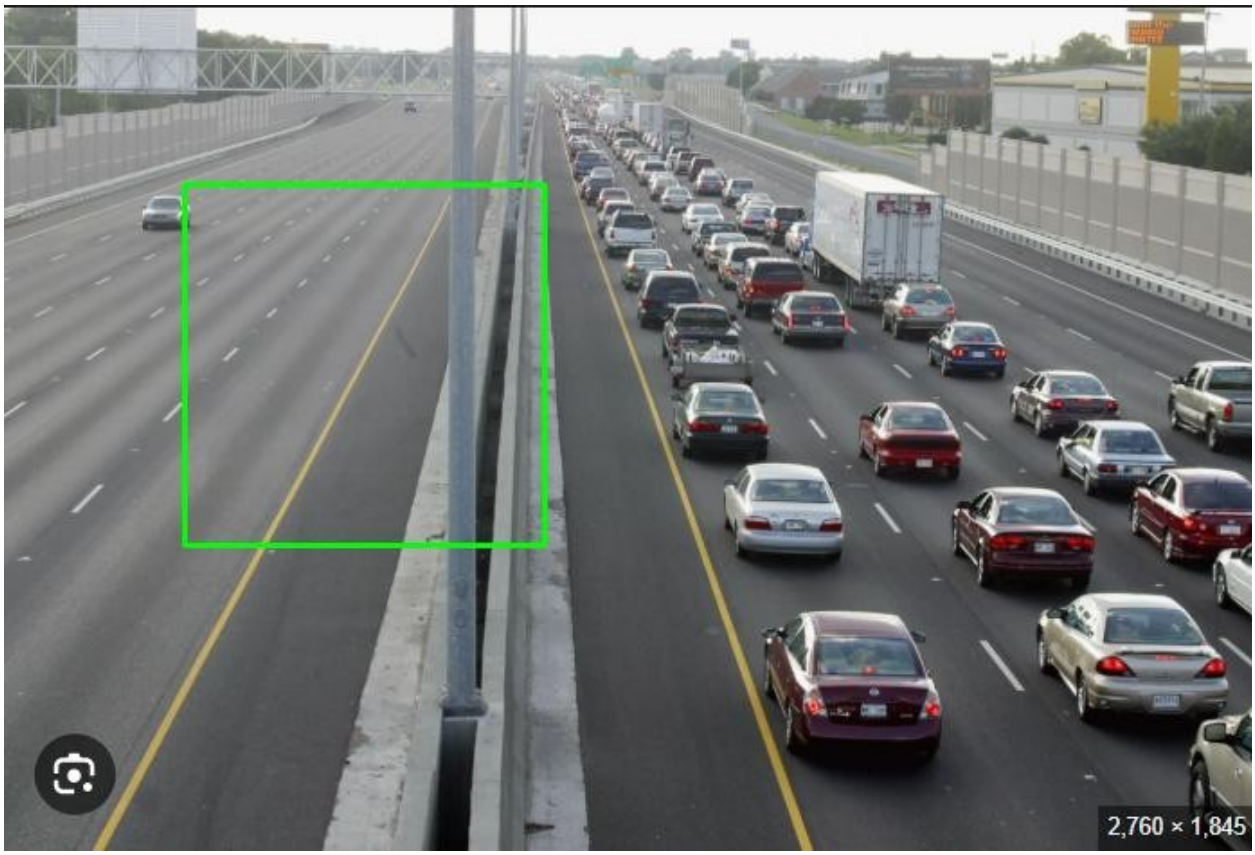
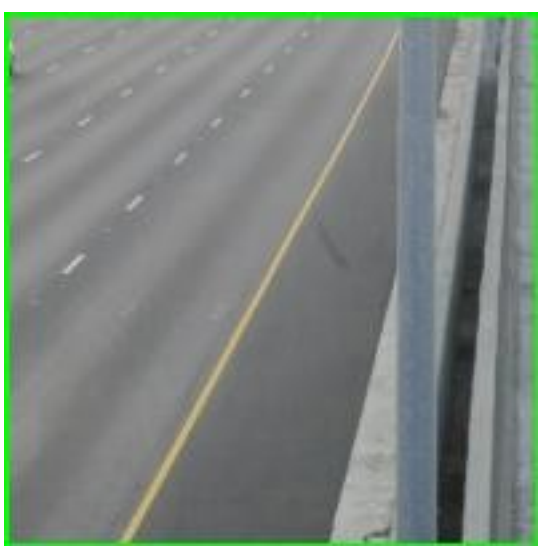
```
import cv2
from google.colab.patches import cv2_imshow

cap = cv2.VideoCapture(list(uploaded.keys())[0])

frames = []

while True:
    ret,frame = cap.read()
    if not ret:
        break
    frames.append(frame)

for frame in reversed(frames):
    cv2_imshow(frame)
```

Output hidden; open in https://colab.research.google.com to view.

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
```

```
face = cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml')

faces = face.detectMultiScale(gray,1.3,5)

for (x,y,w,h) in faces:
    cv2.rectangle(img,(x,y),(x+w,y+h),(0,255,0),2)

cv2_imshow(img)
```



```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread(list(uploaded.keys())[0])

cv2.rectangle(img,(100,100),(300,300),(0,255,0),2)
```

```
object = img[100:300,100:300]

cv2_imshow(img)
cv2_imshow(object)
```